

ERP Academico

Guia Completo para Apresentacao N2

Disciplina: Programacao Web

Tecnologias: React + Firebase (Firestore + Auth)

Grupo: 4 pessoas

DIVISAO DE PARTES PARA APRESENTACAO:

Pessoa 1: Login + Protecao de Rotas + Context API

Pessoa 2: CRUD de Cursos + Firebase (banco.js + config)

Pessoa 3: CRUD de Alunos + CRUD de Matriculas

Pessoa 4: Relatorio (JOIN) + Layout + Componentes

ERP Academico - Guia de Apresentacao N2

PESSOA 1: Login + Protecao de Rotas + Context API

Arquivos que voce explica:

- src/paginas/Login/Login.jsx
- src/contextos/ContextoAutenticacao.jsx
- src/componentes/RotaPrivada/RotaPrivada.jsx
- src/firebase/autenticacao.js

1. Context API (ContextoAutenticacao.jsx)

O que e: Um "provedor" que compartilha o estado de autenticacao com toda a aplicacao sem passar props manualmente.

Como funciona:

- createContext() cria o contexto
- ProvedorAutenticacao envolve toda a app no App.js
- useAutenticacao() e um hook customizado para acessar usuario, estaAutenticado, entrar(), sair()

Ponto-chave: O useEffect com onAuthStateChanged e um listener do Firebase que detecta automaticamente login/logout. Mantem sessao ativa ao recarregar pagina.

2. Tela de Login (Login.jsx)

Funcionalidades:

- Validacao em tempo real (so mostra erro apos primeira tentativa)
- Mostrar/esconder senha
- Se ja logado, redireciona automaticamente
- Mensagem de erro traduzida do Firebase

Perguntas possiveis:

- "Como funciona a validacao?" -> validarEmail() e validarSenha() retornam string de erro ou vazia
- "O que e tentouEnviar?" -> Controla se mostra erros em tempo real (ativa apos primeiro submit)

3. Protecao de Rotas (RotaPrivada.jsx)

Verifica se usuario esta autenticado. Se nao, redireciona para /login. Se sim, renderiza children.

Enquanto verifica (carregando=true), mostra spinner.

Perguntas possiveis:

- "O que acontece se acessar /alunos sem login?" -> Redireciona para /login
- "O que e children?" -> Conteudo dentro da tag <RotaPrivada>...</RotaPrivada>

4. Firebase Auth (autenticacao.js)

Funcoes:

- entrarComEmail(): usa signInWithEmailAndPassword do Firebase
- sairDoSistema(): usa signOut
- enviarRedefinicaoSenha(): envia e-mail de reset

Todas retornam { sucesso: true/false, mensagem }
traduzirErro() converte codigos Firebase para portugueses.

PESSOA 2: CRUD de Cursos + Firebase

Arquivos que voce explica:

- src/firebase/configuracao.js
- src/firebase/banco.js
- src/paginas/Cursos/Cursos.jsx

1. Configuracao Firebase (configuracao.js)

Inicializa conexao com Firebase usando credenciais do .env

- initializeApp() cria instancia
- getAuth() retorna servico de autenticacao
- getFirestore() retorna banco de dados
- Credenciais em .env por seguranca

Pergunta: "Por que .env?" -> Para nao expor chaves no codigo-fonte

2. Funcoes CRUD Genericas (banco.js)

Funcoes reutilizaveis para QUALQUER colecao:

- criarDocumento(colecao, dados): CREATE + timestamp
- buscarTodos(colecao): READ todos os documentos
- buscarPorId(colecao, id): READ um especifico
- atualizarDocumento(colecao, id, dados): UPDATE + timestamp
- excluirDocumento(colecao, id): DELETE

Padrao: Todas retornam { sucesso: true/false, dados/mensagem }

Pergunta: "O que e colecao?" -> Como uma tabela. Temos: alunos, cursos, matriculas

3. CRUD de Cursos (Cursos.jsx)

Estados principais:

- cursos: array com todos os cursos (Read)
- formulario: { nome, carga_horaria, turno } (Create/Update)
- editando: curso sendo editado (null = criando novo)
- excluindo: curso a excluir (modal de confirmacao)

Fluxo CRUD:

- CREATE: formulario -> validar() -> criarDocumento("cursos", dados)
- READ: useEffect -> carregarCursos() -> buscarTodos("cursos")
- UPDATE: abrirEditar() preenche form -> aoSalvar() -> atualizarDocumento()
- DELETE: clica excluir -> modal -> aoExcluir() -> excluirDocumento()

Validacao: Nome >= 3 chars, carga >= 20h, turno obrigatorio

PESSOA 3: CRUD de Alunos + CRUD de Matriculas

Arquivos que voce explica:

- src/paginas/Alunos/Alunos.jsx
- src/paginas/Matriculas/Matriculas.jsx

1. CRUD de Alunos (Alunos.jsx)

Mesma estrutura do CRUD de Cursos, porem:

- Campos: nome, email, idade, curso_id (CHAVE ESTRANGEIRA)
- O aluno e associado a um curso via curso_id
- Validacoes: nome >= 3, email valido (regex), idade 16-100, curso obrigatorio
- Usa TabelaDados com coluna customizada para nome do curso

Chave estrangeira:

- curso_id referencia um documento da colecao "cursos"
- getNomeCurso(cursoid) faz find() no array de cursos para exhibir nome

Pergunta: "Como aluno se conecta ao curso?" -> Pelo campo curso_id

2. CRUD de Matriculas (Matriculas.jsx)

DIFERENCIAL deste CRUD:

- Ao selecionar aluno, curso preenche AUTOMATICAMENTE
- Campo curso e DESABILITADO (nao editavel)
- Ao editar matricula, select de aluno tambem fica desabilitado

Campos: aluno_id, curso_id (automatico), data_matricula, status

Validacao de duplicata: Verifica se aluno ja esta matriculado no mesmo curso.

Como funciona o auto-preenchimento:

```
const aluno = alunos.find(a => a.id === valor);  
setFormulario({ ...form, aluno_id: valor, curso_id: aluno.curso_id })
```

Pergunta: "Por que curso nao pode ser alterado?" -> Aluno ja tem curso definido no cadastro. Matricula apenas registra inscricao.

PESSOA 4: Relatorio (JOIN) + Layout + Componentes

Arquivos que voce explica:

- src/paginas/Relatorio/Relatorio.jsx
- src/componentes/Layout/Layout.jsx
- src/componentes/TabelaDados/TabelaDados.jsx
- src/App.js

1. Relatorio com JOIN (Relatorio.jsx) - MAIS IMPORTANTE

O que e JOIN:

No SQL, JOIN cruza dados de tabelas diferentes. Aqui fazemos com JavaScript:

```
const dadosRelatorio = matriculas.map((matricula) => {  
  const aluno = alunos.find(a => a.id === matricula.aluno_id);  
  const curso = cursos.find(c => c.id === matricula.curso_id);  
  return { nomeAluno: aluno.nome, nomeCurso: curso.nome, ... };  
});
```

Para cada matricula, busca aluno e curso pelo ID e monta objeto unificado.

Funcionalidades extras:

- Filtros por curso e status (.filter())
- Totalizadores (contagem por status)
- Exportar CSV (cria arquivo download)

Pergunta: "O que e find()?" -> Busca primeiro elemento que satisfaz condicao. Equivalente ao WHERE do SQL.

2. Layout (Layout.jsx)

Define estrutura: barra lateral (menu) + area de conteudo.

- <Outlet /> renderiza pagina ativa dentro do layout
- NavLink aplica classe CSS quando rota esta ativa
- Modal de confirmacao de logout
- Exibe nome do usuario (vem do contexto)

Pergunta: "O que e Outlet?" -> Onde rotas filhas sao renderizadas (definido no App.js)

3. TabelaDados (TabelaDados.jsx) - Componente Reutilizavel

Evita repetir codigo de tabela em 3 paginas.

Props:

- colunas: [{ chave, titulo, renderizar? }]
- dados: array de objetos
- aoEditar / aoExcluir: funcoes de callback

Se coluna tem renderizar(), usa ela. Senao, exibe valor direto.

Pergunta: "O que e componente reutilizavel?" -> Componente generico usado em varias paginas com dados diferentes

ERP Academico - Guia de Apresentacao N2

via props.

4. App.js (Rotas)

- /login -> publica
- / -> protegida (RotaPrivada + Layout)
 - /alunos, /cursos, /matriculas, /relatorio
- /* -> redireciona para /

ProvedorAutenticacao envolve TUDO.

CONCEITOS-CHAVE (para qualquer pessoa do grupo)

useState

Hook para criar variaveis de estado. Quando muda, componente re-renderiza.

```
const [nome, setNome] = useState("")
```

useEffect

Executa codigo ao montar componente ou quando dependencias mudam.

Array vazio [] = executa apenas uma vez.

Context API

Compartilha estado global sem prop drilling. Provider envolve app, hook consome dados.

map() e find()

map(): Transforma cada item do array. Usado para renderizar listas e fazer JOIN.

find(): Busca primeiro item que atende condicao. Equivalente ao WHERE do SQL.

Chave Estrangeira

Campo que referencia ID de outra colecao:

- aluno.curso_id -> colecao "cursos"

- matricula.aluno_id -> colecao "alunos"

Props

Dados passados de componente pai para filho. TabelaDados recebe colunas, dados, etc.

CSS Modules

Arquivos .module.css isolam estilos por componente. Cada classe fica unica no build.

async/await

Para operacoes assincronas (Firebase). await espera resposta. Funcao precisa ser async.

ERP Academico - Guia de Apresentacao N2

POSSIVEIS PERGUNTAS DO PROFESSOR

P: Por que usaram Firebase?

R: Persistencia em nuvem, autenticao segura e banco NoSQL. localStorage so persiste no navegador local.

P: O que e Firestore?

R: Banco NoSQL do Firebase. Organizado em colecoes (tabelas) e documentos (linhas). Cada documento tem ID unico.

P: Como funciona protecao de rotas?

R: RotaPrivada verifica estaAutenticado. Se false, redireciona. Todas rotas protegidas estao dentro dele no App.js.

P: O que e componente reutilizavel?

R: Componente generico usado em varias paginas. TabelaDados recebe configuracao via props.

P: Como funciona o JOIN?

R: map() itera matriculas, find() busca aluno e curso pelo ID. Simula JOIN de SQL em JavaScript.

P: O que e Context API?

R: Mecanismo para compartilhar dados globalmente. Provider envolve app, hook consome.

P: Por que curso e automatico na matricula?

R: Aluno ja tem curso no cadastro. Ao selecionar aluno, buscamos curso_id automaticamente.

P: O que e useEffect?

R: Hook para efeitos colaterais. Carrega dados ao montar. Array [] = executa uma vez.

P: Diferenca entre estado e props?

R: Estado e interno e mutavel. Props vem do pai e sao somente leitura.

P: O que e o Outlet?

R: Componente do React Router que renderiza a rota filha ativa dentro do layout pai.

ESTRUTURA DE PASTAS

src/

App.js -> Rotas da aplicacao
index.js -> Ponto de entrada
index.css -> Estilos globais

firebase/

configuracao.js -> Conexao Firebase
autenticacao.js -> Login, logout, reset
banco.js -> CRUD generico (Firestore)

contextos/

ContextoAutenticacao.jsx -> Context API (login global)

componentes/

Layout/Layout.jsx -> Barra lateral + conteudo
RotaPrivada/RotaPrivada.jsx -> Protecao de rotas
TabelaDados/TabelaDados.jsx -> Tabela reutilizavel

paginas/

Login/Login.jsx -> Tela de login
PainellInicial/ -> Dashboard
Cursos/Cursos.jsx -> CRUD 1
Alunos/Alunos.jsx -> CRUD 2
Matriculas/Matriculas.jsx -> CRUD 3
Relatorio/Relatorio.jsx -> Relatorio com JOIN

Fluxo da Aplicacao:

1. Acessa / -> RotaPrivada verifica login -> Nao logado vai para /login
2. Faz login -> Firebase valida -> onAuthStateChanged atualiza contexto
3. Navega -> Layout fixo, Outlet renderiza pagina ativa
4. CRUD -> banco.js -> Firestore persiste
5. Relatorio -> Carrega 3 colecoes, cruza com map+find (JOIN)